

[🏠](#) » [Quick reference for the ESP8266](#) » [MicroPython tutorial for ESP8266](#) »

1. Getting started with MicroPython on the ESP8266

This is the documentation for the latest development branch of MicroPython and may refer to features that are not available in released versions.

If you are looking for the documentation for a specific release, use the drop-down menu on the left and select the desired version.

1. Getting started with MicroPython on the ESP8266

Using MicroPython is a great way to get the most of your ESP8266 board. And vice versa, the ESP8266 chip is a great platform for using MicroPython. This tutorial will guide you through setting up MicroPython, getting a prompt, using WebREPL, connecting to the network and communicating with the Internet, using the hardware peripherals, and controlling some external components.

Let's get started!

1.1. Requirements

The first thing you need is a board with an ESP8266 chip. The MicroPython software supports the ESP8266 chip itself and any board should work. The main characteristic of a board is how much flash it has, how the GPIO pins are connected to the outside world, and whether it includes a built-in USB-serial convertor to make the UART available to your PC.

The minimum requirement for flash size is 1Mbyte. There is also a special build for boards with 512KB, but it is highly limited comparing to the normal build: there is no support for filesystem, and thus features which depend on it won't work (WebREPL, mip, etc.). As such, 512KB build will be more interesting for users who build from source and fine-tune parameters for their particular application.

Names of pins will be given in this tutorial using the chip names (eg GPIO0) and it should be straightforward to find which pin this corresponds to on your particular board.

1.2. Powering the board

If your board has a USB connector on it then most likely it is powered through this when connected to your PC. Otherwise you will need to power it directly. Please refer to the documentation for your board for further details.

1.3. Getting the firmware

The first thing you need to do is download the most recent MicroPython firmware .bin file to load onto your ESP8266 device. You can download it from the [MicroPython downloads page](#). From here, you have 3 main choices

- Stable firmware builds for 1024kb modules and above.
- Daily firmware builds for 1024kb modules and above.
- Daily firmware builds for 512kb modules.

If you are just starting with MicroPython, the best bet is to go for the Stable firmware builds. If you are an advanced, experienced MicroPython ESP8266 user who would like to follow development closely and help with testing new features, there are daily builds (note: you actually may need some development experience, e.g. being ready to follow git history to know what new changes and features were introduced).

Support for 512kb modules is provided on a feature preview basis. For end users, it's recommended to use modules with flash of 1024kb or more. As such, only daily builds for 512kb modules are provided.

1.4. Deploying the firmware

Once you have the MicroPython firmware (compiled code), you need to load it onto your ESP8266 device. There are two main steps to do this: first you need to put your device in boot-loader mode, and second you need to copy across the firmware. The exact procedure for these steps is highly dependent on the particular board and you will need to refer to its documentation for details.

If you have a board that has a USB connector, a USB-serial convertor, and has the DTR and RTS pins wired in a special way then deploying the firmware should be easy as all steps can be done automatically. Boards that have such features include the Adafruit Feather HUZZAH and NodeMCU boards.

If you do not have such a board, you need keep GPIO0 pulled to ground and reset the device by pulling the reset pin to ground and releasing it again to enter programming mode.

For best results it is recommended to first erase the entire flash of your device before putting on

new MicroPython firmware.

Currently we only support esptool.py to copy across the firmware. You can find this tool here: <https://github.com/espressif/esptool/>, or install it using pip:

```
pip install esptool
```

Versions starting with 1.3 support both Python 2.7 and Python 3.4 (or newer). An older version (at least 1.2.1 is needed) works fine but will require Python 2.7.

Any other flashing program should work, so feel free to try them out or refer to the documentation for your board to see its recommendations.

Using esptool.py you can erase the flash with the command:

```
esptool.py --port /dev/ttyUSB0 erase_flash
```

And then deploy the new firmware using:

```
esptool.py --port /dev/ttyUSB0 --baud 460800 write_flash --flash_size=detect 0  
esp8266-20170108-v1.8.7.bin
```

You might need to change the “port” setting to something else relevant for your PC. You may also need to reduce the baudrate if you get errors when flashing (eg down to 115200). The filename of the firmware should also match the file that you have.

For some boards with a particular FlashROM configuration (e.g. some variants of a NodeMCU board) you may need to manually set a compatible [SPI Flash Mode](#). You’d usually pick the fastest option that is compatible with your device, but the `-fm dout` option (the slowest option) should have the best compatibility:

```
esptool.py --port /dev/ttyUSB0 --baud 460800 write_flash --flash_size=detect -fm dout 0  
esp8266-20170108-v1.8.7.bin
```

If the above commands run without error then MicroPython should be installed on your board!

If you pulled GPIO0 manually to ground to enter programming mode, release it now and reset the device by again pulling the reset pin to ground for a short duration.

1.5. Serial prompt

Once you have the firmware on the device you can access the REPL (Python prompt) over UART0 (GPIO1=TX, GPIO3=RX), which might be connected to a USB-serial convertor, depending on your board. The baudrate is 115200. The next part of the tutorial will discuss the prompt in more detail.

1.6. WiFi

After a fresh install and boot the device configures itself as a WiFi access point (AP) that you can connect to. The ESSID is of the form MicroPython-xxxxxx where the x's are replaced with part of the MAC address of your device (so will be the same everytime, and most likely different for all ESP8266 chips). The password for the WiFi is micropython (note the upper-case N). Its IP address will be 192.168.4.1 once you connect to its network. WiFi configuration will be discussed in more detail later in the tutorial.

1.7. Troubleshooting installation problems

If you experience problems during flashing or with running firmware immediately after it, here are troubleshooting recommendations:

- Be aware of and try to exclude hardware problems. There are 2 common problems: bad power source quality and worn-out/defective FlashROM. Speaking of power source, not just raw amperage is important, but also low ripple and noise/EMI in general. If you experience issues with self-made or wall-wart style power supply, try USB power from a computer. Unearthed power supplies are also known to cause problems as they source of increased EMI (electromagnetic interference) - at the very least, and may lead to electrical devices breakdown. So, you are advised to avoid using unearthed power connections when working with ESP8266 and other boards. In regard to FlashROM hardware problems, there are independent (not related to MicroPython in any way) reports ([e.g.](#)) that on some ESP8266 modules, FlashROM can be programmed as little as 20 times before programming errors occur. This is *much* less than 100,000 programming cycles cited for FlashROM chips of a type used with ESP8266 by reputable vendors, which points to either production rejects, or second-hand worn-out flash chips to be used on some (apparently cheap) modules/boards. You may want to use your best judgement about source, price, documentation, warranty, post-sales support for the modules/boards you purchase.
- The flashing instructions above use flashing speed of 460800 baud, which is good compromise between speed and stability. However, depending on your module/board, USB-

UART convertor, cables, host OS, etc., the above baud rate may be too high and lead to errors. Try a more common 115200 baud rate instead in such cases.

- If lower baud rate didn't help, you may want to try older version of esptool.py, which had a different programming algorithm:

```
pip install esptool==1.0.1
```

This version doesn't support `--flash_size=detect` option, so you will need to specify FlashROM size explicitly (in megabits). It also requires Python 2.7, so you may need to use `pip2` instead of `pip` in the command above.

- The `--flash_size` option in the commands above is mandatory. Omitting it will lead to a corrupted firmware.
- To catch incorrect flash content (e.g. from a defective sector on a chip), add `--verify` switch to the commands above.
- Additionally, you can check the firmware integrity from a MicroPython REPL prompt (assuming you were able to flash it and `--verify` option doesn't report errors):

```
import esp
esp.check_fw()
```

If the last output value is True, the firmware is OK. Otherwise, it's corrupted and need to be reflashed correctly.

- If you experience any issues with another flashing application (not esptool.py), try esptool.py, it is a generally accepted flashing application in the ESP8266 community.
- If you still experience problems with even flashing the firmware, please refer to esptool.py project page, <https://github.com/espressif/esptool> for additional documentation and bug tracker where you can report problems.
- If you are able to flash firmware, but `--verify` option or `esp.check_fw()` return errors even after multiple retries, you may have a defective FlashROM chip, as explained above.